

APPLICATION FLOW

TalentLock Employer & Freelancer Journeys

A complete walkthrough of both user journeys — from sign-up through exclusive booking, rate negotiation, AI-generated agreements and e-signing — plus the full catalogue of platform features.

Secure Freelancer Booking Platform

Generated 12 July 2026

The Core Concept

TalentLock is an exclusive freelancer booking platform. Unlike a job board, a freelancer is locked to a single employer for the duration of an engagement — becoming unavailable to everyone else. Every journey converges on one handshake.

Shared entry point (both roles)

- 1 Landing & authentication**
Public landing at /. Sign up / sign in via Clerk. New users get a 404 from GET /api/users/me and are routed to onboarding.
- 2 Onboarding & role choice**
Pick Freelancer or Employer — this branches the entire product. Profiles are created (users + freelancer_profiles or employer_profiles).
- 3 Role-specific dashboard**
Both land on /dashboard with metrics tailored to their role.

The convergence — booking & agreement state machine

- pending / negotiating** employer proposes first
| accept / counter (turn-based) until...
- rate: AGREED** unlocks agreement generation
| GPT-4 drafts contract
- draft -> redlined -> partially_signed** each party e-signs
| both signatures captured
- fully_signed -> booking ACTIVE** freelancer LOCKED / exclusive
| work delivered
- completed -> review** availability freed, rating recorded

Key gate: the AI agreement cannot be generated while negotiationStatus === 'negotiating'. Rate must be mutually agreed first. When both parties sign, the booking automatically transitions to 'active' and the freelancer becomes exclusive.

The Employer Journey

Goal: find the right talent, lock them in exclusively, and formalise the engagement legally — with AI assistance at every decision point.

Talent Vault

AI Match

TalentSearch

Bookings

Agreements

Spend & Hiring Analytics

Teams

1

Sign up & onboard

Register via Clerk, choose the Employer role, enter company name, industry, size and description. Creates users + employer_profiles.

2

Find talent — 3 ways

Talent Vault (browse/filter by field, rate, availability, keyword); AI Match chat (GPT recommends + explains fit); TalentSearch (AI auto-scouts profiles for you).

3

Post a job (optional)

Create a job requirement with the AI Job Description Assistant (generate / improve / check). Posting triggers freelancers' Cruise Mode evaluations. Plan-limited per month.

4

Request a discovery meeting (optional)

On confirmation, an AI Meeting Brief is generated for you: candidate snapshot, why-they-match, tailored questions, rate context and watch points.

5

Initiate a booking

The pivotal action. A SELECT ... FOR UPDATE quota check runs inside a transaction; over-limit returns 402 and routes to /pricing. Booking opens as pending / negotiating.

6

Negotiate the rate

Turn-based accept / counter. A Rate Suggestion widget shows market median plus your historical average. You proposed first; freelancer responds.

7

Generate the agreement

Blocked until the rate is AGREED. GPT-4 drafts an industry-specific contract. Growth+ unlocks AI Redlining and a Contract Health Score (0-100, 5 dimensions).

8

E-sign

Sign with an uploaded signature image or typed name. Signing role is derived server-side (IDOR-safe). When both sign, the agreement is fully_signed.

9

Engagement goes live

Booking flips to active and the freelancer is locked/exclusive. A certified PDF of the executed agreement is unlocked for both parties.

10

Close out & review

Track milestones, mark completed (frees availability), then leave a 1-5 star review + comment that feeds the freelancer's public rating.

11

Measure

Dashboard Spend Analytics (spend trend, top freelancers, rate vs market) and Hiring Analytics (funnel, skills gap, retention, lifecycle). Enterprise adds Team accounts.

The Freelancer Journey

Goal: get discovered, get booked on great terms, deliver, and build a verifiable reputation — with AI doing the heavy lifting on discovery and paperwork.

Resume Auto-build

Verification

Availability

Cruise Mode

Proposals

Agreements

Earnings Intelligence

1

Sign up & onboard

Register via Clerk, choose Freelancer, then pick a profession category (Technology or Education). Education adds teaching-specific fields.

2

Auto-build from resume

Upload a resume and AI extracts tagline, field, skills, experience and rate — instantly creating your profile. Or fill the form manually.

3

Complete the profile

Skills, rate, field, bio, photo, availability. completenessScore recalculates on every save. Gate: $\geq 60\%$ complete to appear in the Talent Vault at all.

4

Verify identity

Upload ID / credential documents for AI review to earn a verified badge that employers can see.

5

Manage availability

Mark holidays and unavailable date ranges on a calendar. Confirmed bookings auto-create availability blocks.

6

Get discovered proactively

Cruise Mode (Pro): set rules; AI evaluates every new job post and auto-pitches matching employers on your behalf (10/mo, dry-run, activity feed). Plus inbound TalentSearch interest.

7

Receive a booking request

The employer initiates. You get an in-app notification + email. Booking starts pending / negotiating.

8

Negotiate the rate

Accept or counter (turn-based; you can't act twice in a row). An AI Proposal Generator drafts your response; a Rate Suggestion widget gives market context.

9

Understand & sign

Once the rate is agreed, use the freelancer-only 'Summarise for me' AI (6 plain-English sections + attention flags) before signing with your saved signature.

10

Go exclusive

When both parties sign, the engagement is live: you become locked (unavailable, Lock badge) to everyone else for the engagement window.

11

Deliver, complete, get reviewed

Track milestones; on completion your availability frees up and the employer can review you (you can reply).

12

Track earnings

Dashboard Earnings Intelligence: 6-month trend vs platform average, rate percentile benchmark, monthly projection and top-earning skills.

Full Feature Catalogue (1 / 2)

Every capability shipped across the platform. Features 01-20.

01

Dual Role System

Register as freelancer or employer; onboarding tailors the whole experience per role.

02

Talent Vault

Browse/filter freelancers by field, rate, availability, available-from date and keyword; shortlist with a heart; $\geq 60\%$ completeness gate.

03

Exclusive Bookings

Booked freelancers get a Lock badge and become unavailable to all other employers.

04

Rate Negotiation

Turn-based propose/accept/counter; agreement generation gated until both agree.

05

AI Talent Matching

GPT chat that recommends matching freelancers for an employer's need.

06

AI Agreement Generation

GPT-4 drafts legal engagement contracts from booking details + templates.

07

Signature Upload

Upload a handwritten signature image once; reused across all agreements.

08

Agreement Safe Locker

Fully executed agreements unlock a certified download per party.

09

Milestone Tracking

Bookings track milestones with title, amount, due date and status.

10

Reviews & Ratings

Employers rate completed bookings 1-5 + comment; freelancers can reply.

11

Portfolio

Freelancers manage portfolio items with images, URLs and tags.

12

Public Profiles

Unauthenticated `/f/:id` pages with a read-only availability calendar.

13

Availability Calendar

Visual date-range blocks (booked/holiday/unavailable); auto-blocks from bookings; employer read-only view; next-available badge.

14

Subscription Tiers

5 plans with enforced limits on bookings, job posts and interests.

15

Admin Console

Stats, users, audit log, booking/job overview; separate HMAC auth + CSRF.

16

Document Verification

AI review of uploaded ID/credentials; verified badge to employers.

17

AI Token Tracking

Monthly token quota per employer plan; per-conversation breakdown (Growth+).

18

Agreement Templates

Industry-specific templates + enterprise custom clauses for generation.

19

Contract Redlining

AI review and edit suggestions before signing (Growth+).

20

Job Description Assistant

AI writing assistant (generate / improve / check) for job posts.

Full Feature Catalogue (2 / 2)

Features 21-38, followed by the subscription tiers that gate them.

21

Smarter Matching

AI explains why each freelancer was recommended (skills, rate, availability).

22

Notifications Centre

In-app bell + unread badge; 15 event triggers; 30s polling; optional email.

23

Earnings Intelligence

Freelancer: 6-month trend vs avg, rate percentile, projection, top skills.

24

Employer Spend Analytics

Spend trend, field breakdown, top freelancers, committed spend, rate vs market.

25

Hiring Analytics

Funnel conversion, skills demand vs supply gap, retention, lifecycle, outcomes.

26

Security Hardening

Helmet, body limits, Pino redaction, text sanitisation, admin CSRF, audit logs, GDPR deletion.

27

Product Gaps polish

Email notifications (opt-out), keyword search, completeness gate, pagination, booking message, public preview.

28

AI Proposal Generator

Freelancer 'Write proposal' drawer on pending bookings; three tones; copyable block.

29

Smart Rate Suggestions

Rate context widget on booking + negotiation; static (all plans) + AI (Growth+).

30

Team Accounts (Enterprise)

Invite members, shared shortlist, team-level analytics; role-based permissions.

31

AI Contract Health Score

On-demand 0-100 score across 5 dimensions with A-F grade; cached; both parties.

32

Auth Hardening

Per-resource authorization on 11 routes + storage ACL; IDOR protection (401/403/404).

33

Agreement AI Summary

Freelancer-only 6-section plain-English summary + attention flags; disclaimer first.

34

Agreement PDF Download

Formatted PDF with signatures, timestamps, metadata; cached in GCS; both parties.

35

Cruise Mode

Freelancer_pro: AI evaluates every new job vs rules and auto-pitches employers; dry-run; 10/mo quota.

36

Teaching Professional Profile

Profession category + rate types + education fields (subjects, degree, licence, DBS, research).

37

TalentSearch (Employer Cruise Mode)

AI auto-evaluates freelancer profiles vs employer rules and sends Express Interest notifications.

38

AI Meeting Brief Generator

On meeting confirm, AI 5-section brief for the employer; cached; regeneratable (in active development).

Subscription tiers

Freelancer Free — \$0	3 active bookings, 5 interests/mo, listed in Vault
Freelancer Pro — \$19/mo	10 active bookings, unlimited interests, Cruise Mode, Pro badge
Employer Starter — \$49/mo	3 active bookings, 5 job posts/mo, 50k AI tokens/mo
Employer Growth — \$199/mo	15 active bookings, 30 job posts/mo, 250k tokens, redlining + health score
Employer Enterprise — Custom	Unlimited; team accounts + team analytics enabled

Cross-cutting layers apply to both roles: Notifications (in-app + email), Subscription gating (402 with planNeeded), the separate Admin console, and security hardening (per-resource auth, sanitisation, audit logs, GDPR deletion).

Architecture & Tech Stack

A pnpm monorepo. The Vite dev server proxies all /api calls to the Express server on port 8080, so the whole app runs behind a single URL.

Technology by layer

Frontend	React 19 + Vite 7, TypeScript, Wouter routing, TanStack Query, Tailwind CSS + Radix UI, Recharts, Framer Motion. Served on port 25807.
Backend	Express 5 (Node, ESM) bundled with esbuild, Pino structured logging, Helmet + CORS. REST API on port 8080 under /api.
Database	PostgreSQL (Neon) via Drizzle ORM. 25 schema files; migrations pushed with drizzle-kit. No raw SQL in app code.
Auth	Clerk (user auth, sessions, account deletion). Admin console uses a separate HMAC-signed cookie (tl_admin, 8h) + csrf-csrf double-submit.
AI	OpenAI (GPT) via a shared server integration package. Per-employer monthly token quota tracked in token_usage per feature label.
Storage	Google Cloud Storage for signatures, documents and cached agreement PDFs (presigned upload URLs). Local fallback dir for dev.
Email	Resend (transactional). No-op when RESEND_API_KEY is unset. Fire-and-forget on notification events.
PDF	@react-pdf/renderer server-side for certified, formatted agreement downloads (cached in GCS).
Contract	OpenAPI (lib/api-spec/openapi.yaml) is the single source of truth. Orval generates React Query hooks (api-client-react) + Zod schemas (api-zod).

Monorepo layout

artifacts/talentlock	React + Vite frontend (27 pages, ~119 components)
artifacts/api-server	Express 5 API server (35 route modules)
lib/db	Drizzle ORM schema + PostgreSQL migrations
lib/api-spec	OpenAPI spec + Orval codegen config
lib/api-client-react	Generated React Query hooks (never hand-edited)
lib/api-zod	Generated Zod validation schemas (never hand-edited)
lib/integrations-openai-*	OpenAI clients (server + react)

API workflow: edit openapi.yaml -> run Orval codegen -> implement the route -> register it -> use the generated hook on the frontend. Generated hooks (api-client-react) and Zod schemas (api-zod) are never hand-edited.

API Endpoint Reference

All routes are prefixed /api and (unless noted public) require Clerk auth with per-resource authorization:
401 unauthenticated, 403 non-participant, 404 unknown id.

Users & Account

GET	/api/users/me	Current user profile
PUT	/api/users/me	Update profile
PUT	/api/users/me/signature	Save/clear signature image
PATCH	/api/users/me/notification-preferences	Toggle email opt-in
POST	/api/account/delete-request	GDPR deletion request
POST	/api/demo/sign-in-token	Mint demo Clerk token (dev)

Freelancers & Profiles

GET	/api/freelancers	List (filters, ?q=, ?availableFrom=, >=60%)
GET	/api/freelancers/:id	Freelancer detail
GET/PUT	/api/freelancers/me	My profile (recalcs completeness)
GET/PUT	/api/employers/me	My employer profile
GET	/api/availability/:freelancerId	Public availability blocks
GET/POST	/api/availability/me	My blocks / create block
DELETE	/api/availability/me/:id	Delete manual block

Jobs & Bookings

GET/POST	/api/job-requirements	List / create job (triggers Cruise Mode)
GET/PATCH/DELETE	/api/job-requirements/:id	Detail / update / delete
GET/POST	/api/bookings	Paginated list / create (quota-checked)
GET/PATCH	/api/bookings/:id	Detail / update status + milestones
POST	/api/bookings/:id/negotiate	Accept / counter rate

Agreements

GET/POST	/api/agreements	List / generate (AI, gated on agreed rate)
GET	/api/agreements/:id	Agreement detail
POST	/api/agreements/:id/sign	Sign (role derived server-side)
GET	/api/agreements/:id/download	Certified PDF (fully_signed only)
POST	/api/agreements/:id/redline	AI redline suggestions (Growth+)
PATCH	/api/agreements/:id/accept-redline	Accept a redline
POST	/api/agreements/:id/health-score	AI 0-100 quality score
POST	/api/agreements/:id/summarise	AI summary (freelancer-only)

Meetings & AI

GET/POST	/api/meetings	Paginated list / request meeting
GET/PATCH	/api/meetings/:id	Detail / update (confirm triggers brief)
POST	/api/meetings/:id/brief	Generate/regenerate AI brief (202)
GET/POST	/api/openai/conversations	AI match chat
GET	/api/ai/match-explanation	Why-they-match reasons
POST	/api/ai/job-description	JD assistant (generate/improve/check)
POST	/api/ai/proposal	Freelancer proposal generator
POST	/api/ai/interview-questions	Interview question generator
POST	/api/ai/rate-suggestion	Rate recommendation (AI = Growth+)

Reviews, Notifications, Tokens

GET/POST	/api/reviews (+ /freelancer/:id, /:id/reply)	Reviews + freelancer reply
GET	/api/notifications	Paginated notifications
GET	/api/notifications/unread-count	Unread badge count
PATCH	/api/notifications/read-all /:id/read	Mark read
GET	/api/token-usage/me	Monthly token usage summary
GET	/api/token-usage/conversation/:id	Per-conversation (Growth+)

Cruise Mode & TalentSearch

GET/POST	/api/cruise-mode	Config (freelancer)
PATCH	/api/cruise-mode/{activate dry-run pause deactivate}	State control
POST	/api/cruise-mode/parse-rules	AI parse free-form rules
GET	/api/cruise-mode/activity /stats	Activity feed + stats
GET/POST	/api/talent-search	Config (employer)
PATCH	/api/talent-search/{activate dry-run deactivate}	State control
GET	/api/talent-search/activity /stats	Activity feed + stats

Dashboards, Teams, Subscriptions, Admin

GET	/api/dashboard/stats /activity	Metrics + activity feed
GET	/api/dashboard/earnings-intelligence	Freelancer analytics
GET	/api/dashboard/spend-analytics /hiring-analytics	Employer analytics
GET/POST/P	/api/team (+ /invite, /members, /shortlist)	Enterprise teams
GET	/api/subscriptions/plans /me	Plans + my plan/usage
POST	/api/subscriptions/upgrade	Upgrade (simulated checkout)
GET	/api/admin/{stats users audit bookings jobs ...}	Admin console (HMAC + CSRF)

Per-Feature Plan Gating

Limits are enforced server-side with `SELECT ... FOR UPDATE` quota checks inside a transaction. Values below are the live values from `plans.ts` (source of truth).

Plan limits

Limit	FL Free	FL Pro	Emp Starter	Emp Growth	Emp Enterprise
Active bookings	1	5	2	10	Unlimited
Job posts / month	-	-	5	Unlimited	Unlimited
Express Interests / mo	3	Unlimited	-	-	-
Team seats	1	1	1	3	Unlimited
AI tokens / month	none	none	50k	250k	Unlimited
Price / month	\$0	\$19	\$49	\$199	Custom

Feature gating & failure behaviour

Capability / Action	Requires	When blocked
Cruise Mode (auto-pitch)	freelancer_pro	402 PLAN_LIMIT -> /pricing
Create booking (over limit)	activeBookings quota	402 PLAN_LIMIT -> /pricing
Post job (over limit)	monthlyJobPosts quota	402 PLAN_LIMIT -> /pricing
Express Interest (over limit)	monthlyExpressInterests	402 PLAN_LIMIT -> /pricing
AI rate suggestion	employer_growth+	Static rate data still shown
AI job description assistant	employer_growth+	402 PLAN_LIMIT (AI portion)
Contract redlining	employer_growth+	402 PLAN_LIMIT
Per-conversation token breakdown	employer_growth+	402 PLAN_LIMIT
Any AI feature (match, proposal...)	within monthly token quota	402 TOKEN_LIMIT (inline error)
Team accounts + analytics	employer_enterprise	402 PLAN_LIMIT
Agreement PDF download	all plans (fully_signed)	403 NOT_FULLY_SIGNED otherwise
Agreement AI summary	freelancer-only	403 for employers

Two distinct 402 codes: `PLAN_LIMIT` (hard cap / feature not on plan) redirects to `/pricing`; `TOKEN_LIMIT` (monthly AI budget exhausted) shows an inline error and never redirects. Both return `{ error, code, planNeeded }`.

Database Schema Reference

PostgreSQL via Drizzle ORM (25 schema files in lib/db/src/schema/). Profiles carry both clerkId and userId; access-control helpers join on userId. Bookings use profile IDs; token_usage / notifications use users.id.

Table	Purpose
users	Core accounts linked to Clerk IDs; signature URL, email-notification opt-in
freelancer_profiles	Professional info, skills, rate, availability; averageRating, reviewCount, nextAvailableDate, completenessScore, professionCategory + 12 education fields, profileMatchHash
employer_profiles	Employer company info
job_requirements	Job postings; professionCategory, rateType
bookings	Exclusive engagements; proposedRate, lastProposedBy, negotiationStatus, employer message
agreements	AI legal agreements; both signature URLs + signedAt, status, healthScore(+detail), freelancerSummary
conversations / messages	AI match chat sessions + individual messages
meetings	Discovery meeting requests; briefContent (jsonb), briefGeneratedAt
subscriptions	Per-user billing plan and status
audit_logs	Login/logout + sensitive-action trail; ip, userAgent, entity, metadata
token_usage	AI token consumption per user, per feature, per conversation
documents	Freelancer identity/credential uploads for AI verification
reviews	Employer reviews of completed bookings (one per booking) + reply
notifications	In-app notification rows; server-triggered on key events
account_deletion_requests	GDPR deletion requests (pending -> complete)
availability_blocks	Freelancer unavailability ranges; auto from bookings or manual
milestones	Booking milestones (title, amount, due date, status)
job_interests	Freelancer Express Interest pitches (monthly quota)
portfolio_items	Freelancer portfolio (images, URLs, tags)
saved_freelancers	Employer shortlist (heart icon)
teams / team_members / team_shortlist	Enterprise team accounts, membership + roles, shared shortlist
cruise_mode_configs / _activity	Freelancer auto-pitch rules + per-job evaluation log
talent_search_configs / _activity	Employer auto-scout rules + per-freelancer evaluation log

Feature Delivery Status

26 features tracked in specs/spec.md. 20 are marked Complete (validated); 6 are marked Ready to Execute in the spec index — note their routes are already registered and functional in the running codebase, so the spec status trails the implementation.

Feature	Spec status
AI Token Consumption Dashboard	COMPLETE
AI Enhancements	COMPLETE
Document Verification	COMPLETE
Smarter Matching Explanation	COMPLETE
Agreement Templates + Redlining	COMPLETE
Job Description Assistant	COMPLETE
Per-Conversation Token Breakdown	COMPLETE
Reviews & Ratings	COMPLETE
Notifications Centre	COMPLETE
Earnings Intelligence	COMPLETE
Employer Spend Analytics	COMPLETE
Employer Analytics Dashboard	COMPLETE
Availability Calendar (Visual)	COMPLETE
Security Hardening	COMPLETE
Product Gaps	COMPLETE
AI Proposal Generator	COMPLETE
Smart Rate Suggestions	COMPLETE
Team Accounts (Enterprise)	COMPLETE
AI Contract Health Score	COMPLETE
Auth Hardening (Access Control)	COMPLETE
Agreement AI Summary	READY
Agreement PDF Download	READY
Cruise Mode	READY
Teaching Professional Profile	READY
TalentSearch (Employer Cruise Mode)	READY
AI Meeting Brief Generator	READY

Security & Production-Readiness Review

Findings from the TalentLock Security & Production Readiness review (2026-06-09). All P0 and P1 items are closed; remaining work is P2 (important, non-blocking) and P3 (production config / enterprise).

Pri	Item	Status
P0	IDOR on 11 routes — accessControl.ts + guards	DONE
P0	Auth-gate storage upload URLs + object ACL (namespace by userId)	DONE
P1	Token breakdown — extend to all 9 AI features	DONE
P1	Apply sanitiseText() to 6 missing free-text fields	DONE
P1	Fix premature availability lock (defer to confirmation)	DONE
P1	Add 4 missing endpoint groups to OpenAPI + fix raw fetch	TODO
P2	Automated tests (Vitest + Supertest) + wire validators to CI	TODO
P2	Fix N+1 on bookings/meetings/agreements list endpoints	TODO
P2	Schema & type hygiene (FKs, as any, tx scope, Zod, split routes)	TODO
P2	Stripe real checkout + webhook signature verification	TODO
P2	AI match history cap + profile caching	TODO
P2	Booking acceptance state machine (freelancer accept/decline)	TODO
P3	Boot guard, CORS lockdown, trust proxy, remove demo route	TODO

P0 = critical (IDOR, storage ACL) — closed & validated. P1 = high (token breakdown, sanitisation, availability lock) — closed; OpenAPI cleanup outstanding. P2 = tests, N+1 perf, Stripe, state machine. P3 = boot guards, CORS lockdown, remove demo route before production.

Spec-Driven Development Process

Every feature follows one folder with six ordered files. Each file feeds the next; the Cursor Agent reads only plan.md + task.md (+ UI.md for frontend), never features.md or clarify.md.

File	Owner	Purpose
1. features.md	Product	Defines the feature: what it does, modules, plan/quota details, explicit non-goals
2. clarify.md	Product + Eng	Audits features.md vs architecture; surfaces blockers and open questions
3. plan.md	Engineering	Binding decisions + exact TypeScript resolving every question. Wins over task.md
4. task.md	Engineering	Ordered implementation tasks with exact file paths, snippets, acceptance criteria
5. UI.md	Eng + Design	Every component, page integration, state variant, copy string, a11y requirement
6. validation.md	Engineering	Phase-by-phase test checklist: API, UI states, security, regression. Gates merge

Execution order & binding rules

- 1 project.md first**
Re-read the architecture reference; reuse existing patterns (FOR UPDATE gating, 402 redirect, UTC-month reset) instead of reinventing them.
- 2 Phases run in order**
Database -> Backend (routes -> OpenAPI -> codegen -> typecheck) -> Frontend -> Admin. Never start a phase before the previous one is confirmed.
- 3 plan.md wins**
If plan.md and task.md conflict, plan.md is authoritative. No clarify.md question may remain open in plan.md.
- 4 Non-goals are binding**
If a non-goal surfaces mid-build, stop and re-open the spec rather than silently expanding scope.
- 5 Validation gates merge**
A feature is not merged until every check in validation.md is ticked and the sign-off table is complete.

Environment & Engineering Conventions

Environment variables

Variable	Req	Purpose
DATABASE_URL	Yes	PostgreSQL (Neon) connection string
CLERK_SECRET_KEY / VITE_CLERK_PUBLISHABLE_KEY	Yes	Clerk auth (backend + frontend)
SESSION_SECRET / CSRF_SECRET	Yes	Admin cookie signing + CSRF double-submit
ADMIN_USERNAME / ADMIN_PASSWORD	Yes	Admin console login (fail-closed if unset)
OPENAI_API_KEY_TALENTLOCK	Yes	GPT calls for all AI features
ENABLE_DEMO_LOGIN + DEMO_*_CLERK_ID	Dev	Mint demo Clerk sessions (never in prod)
ALLOWED_ORIGINS / TRUST_PROXY	Prod	CORS allow-list + reverse-proxy client IP
RESEND_API_KEY / EMAIL_FROM / APP_URL	Optional	Transactional email (no-op if unset)
DEFAULT_OBJECT_STORAGE_BUCKET_ID / *	Optional	GCS for uploads (currently unset -> local fallback)

Engineering conventions

Quota gating	All quota checks use <code>SELECT ... FOR UPDATE</code> inside a Drizzle transaction; over-limit returns <code>402 { error, code, planNeeded }</code> .
Fire-and-forget	<code>Auto-blocks</code> , <code>createNotification()</code> , <code>logAudit()</code> , <code>sendNotificationEmail()</code> and AI evaluations all use <code>.catch()</code> — never awaited from handlers.
Input sanitisation	Every free-text DB write passes through <code>sanitiseText()</code> from <code>lib/sanitise.ts</code> .
Completeness score	<code>calculateCompletenessScore()</code> saved atomically in the same <code>db.update()</code> as the profile change.
Pagination shape	<code>bookings/agreements/meetings</code> lists return <code>{ data, total, page, pageSize, totalPages }</code> — never a plain array.
Route ordering	<code>availability/me</code> before <code>/:freelancerId</code> ; <code>notifications/read-all</code> before <code>/:id/read</code> ; <code>admin/csrf-token</code> before CSRF middleware.
Generated code	Never hand-edit <code>lib/api-client-react</code> or <code>lib/api-zod</code> ; regenerate via Orval codegen from <code>openapi.yaml</code> .
Logging	Use <code>req.log</code> (Pino) in route handlers — never <code>console.log</code> ; secrets are redacted.
GDPR deletion	Anonymise in a Drizzle transaction first, then call Clerk outside it; on Clerk failure, reset request to pending for retry.

Monorepo tooling: `pnpm workspace` (target packages with `--filter @workspace/<name>`). Push schema with `pnpm --filter @workspace/db run push`; regenerate the API client with `pnpm --filter @workspace/api-spec run codegen`; verify with `pnpm run typecheck` before any frontend work.